

How to make Optional Parameters in C# with Examples

In this article, I am going to discuss **How to Make Optional Parameters in C#** with Examples. Please read our previous article, where we discussed [AutoMappers in C#](#). This is also one of the most frequently asked interview questions. So here we will discuss the different options that are available in C# to make the method parameters optional.

What are Optional Parameters in C#?

By default, all parameters of a method are required. But in C# 4.0, the concept of Optional Parameters was introduced that allows developers to declare parameters as optional. That means, if these arguments are not passed, they will be omitted from the execution. Optional parameters are not mandatory.

As the name suggests Optional Parameters are not compulsory parameters, they are Optional. In C#, we can use the optional parameters inside Methods, Constructors, Indexers, and Delegates. Each and every optional parameter in C# contains a default value which is part of its definition. And, if we do not pass any values for the optional parameters, then it takes its default value.

Note: The point that you need to remember is the optional parameters are always defined at the end of the parameter list.

How to make Optional Parameters in C#?

We can make the method parameters optional in C# in many different ways as follows.

1. [Using Parameter Array](#)
2. [Method Overloading](#)
3. [Specify Parameter Defaults](#)
4. [Using OptionalAttribute](#)

So, let us discuss all these options one by one with examples.

Using Parameter Array to Make Optional Parameter in C#:

Let us understand how to make method parameters optional using Parameter Array in C# with an example. Consider the following ADDNumbers method. Here, we are declaring the first and second parameters as integers and the third parameter is a parameter array i.e. params object[].

```
public static void ADDNumbers(int FN, int SN, params object[] restOfTheNumbers)
{
    int result = FN + SN;
    foreach (int i in restOfTheNumbers)
    {
        result += i;
    }
    Console.WriteLine("Total = " + result.ToString());
}
```

The above ADDNumbers method allows the user to add 2 or more numbers. The FN and SN parameters are mandatory parameters whereas the restOfTheNumbers parameter is optional. If the user wants to add just two

numbers, then the user can invoke the method as shown below,

ADDNumbers(10, 20);

On the other hand, if the user wants to add 5 numbers, then the user can invoke the method in the below two ways.

ADDNumbers(10, 20, 30, 40, 50);

or

ADDNumbers(10, 20, new object[]{30, 40, 50});

The Parameter Array must be the last parameter in the formal parameter list. The following function will not be compiled as the parameter array not the last parameter of the parameter list.

```
public static void ADDNumbers(int FN, params object[] restOfTheNumbers, int SN)
{
    int result = FN + SN;
    foreach (int i in restOfTheNumbers)
    {
        result += i;
    }
    Console.WriteLine("Total = " + result.ToString());
}
```

Example to Understand Parameter Array to Make Optional Parameter in C#:

In the below example, we are creating the ADDNumbers method to add two or more integer numbers. In the parameter list of the ADDNumbers method, the first two parameters are mandatory parameter and the last parameter is an optional parameter. And for that optional parameter, we can pass 0 or n number of values. As shown in the below example, we are calling the ADDNumbers method by passing two and five arguments.

```
using System;
namespace OptionalParameter
{
    class Program
    {
        static void Main(string[] args)
        {
            ADDNumbers(10, 20);
            ADDNumbers(10, 20, 30, 40);
            ADDNumbers(10, 20, new object[] { 30, 40, 50 });
            Console.ReadLine();
        }
        public static void ADDNumbers(int FN, int SN, params object[] restOfTheNumbers)
        {
            int result = FN + SN;
            foreach (int i in restOfTheNumbers)
            {
                result += i;
            }
            Console.WriteLine("Total = " + result.ToString());
        }
    }
}
```

Output:

```
Total = 30  
Total = 100  
Total = 150
```

Using Method Overloading to Make Optional Parameter in C#:

Let us understand how to make method parameters optional using method overloading in C#. Let's create the following method which will add any number of integers. Here, we created the first two parameters as integers and the third parameter as an integer array. The first two parameters are mandatory and in the case of the third parameter, if you don't want to pass any value, then you simply need to pass null.

```
public static void ADDNumbers(int FN, int SN, int[] restOfTheNumbers)  
{  
    int result = FN + SN;  
    foreach (int i in restOfTheNumbers)  
    {  
        result += i;  
    }  
    Console.WriteLine("Total = " + result.ToString());  
}
```

If you want to add 5 integers, let's say 10, 20, 30, 40, and 50 then you need to call the method as follows.

ADDNumbers(10, 20, new int[]{30, 40, 50});

At the moment all the 3 method parameters are mandatory. Now, if I want to add just 2 numbers, then need to invoke the method as follows.

ADDNumbers(10, 20, null);

Notice that, here I am passing null as the argument for the 3rd parameter. We can make the 3rd parameter optional by overloading ADDNumbers() function which takes two parameters as shown below.

```
public static void ADDNumbers(int FN, int SN)  
{  
    int result = FN + SN;  
    Console.WriteLine("Total = " + result.ToString());  
}
```

Now, we have 2 overloaded versions of the ADDNumbers() function. If we want to add just 2 numbers, then we can use the overloaded version of the ADDNumbers() function which takes 2 parameters as follows.

ADDNumbers(10, 20);

Similarly, if we want to add 3 or more numbers, then we can use the other overloaded version of the ADDNumbers() function which takes 3 parameters as follows.

ADDNumbers(10, 20, new int[] { 30, 40 });

Example to Understand Method Overloading to Make Optional Parameter in C#:

In the below example we are using Method Overloading concept to make method parameters Optional. As you can see, we have created two overloaded version of the ADDNumbers method. One overloaded version takes 2 parameters which is going to add two integer numbers and the other overloaded version takes three parameters going to add more than 2 integers.

```

using System;
namespace OptionalParameter
{
    class Program
    {
        static void Main(string[] args)
        {
            ADDNumbers(10, 20);
            ADDNumbers(10, 20, new int[] { 30, 40, 50 });

            Console.ReadLine();
        }
        public static void ADDNumbers(int FN, int SN, int[] restOfTheNumbers)
        {
            int result = FN + SN;
            foreach (int i in restOfTheNumbers)
            {
                result += i;
            }
            Console.WriteLine("Total = " + result.ToString());
        }
        public static void ADDNumbers(int FN, int SN)
        {
            int result = FN + SN;
            Console.WriteLine("Total = " + result.ToString());
        }
    }
}

```

Output:

```

Total = 30
Total = 150

```

Making Method Parameters Optional by Specifying Parameter Defaults in C#:

Let us understand how to specify the parameter defaults to make the make method parameters optional in C#. We can make the method parameter optional by specifying a default value for the parameter shown below in the below image. As you can see in the below image, we have made the third parameter optional by specifying a default value null. Here, the first and second parameters are mandatory parameters while the third parameter is an optional parameter.

```

public static void ADDNumbers(int FN, int SN, int[] restOfTheNumbers = null)
{
    int result = FN + SN;
    // loop thru restOfTheNumbers only if it is not null otherwise
    // you will get a null reference exception
    if (restOfTheNumbers != null)
    {
        foreach (int i in restOfTheNumbers)
        {
            result += i;
        }
    }
    Console.WriteLine("Total = " + result.ToString());
}

```

As we have specified a default value for the 3rd parameter now it becomes optional. So, if we want to add just 2 numbers, we can invoke the method as follows.

ADDNumbers(10, 20);

On the other hand, if we want to add 3 or more numbers, then we can invoke the method ADDNumbers() as follows.

ADDNumbers(10, 20, new int[] { 30, 40 });

The Optional Parameters in C# must appear after all the Required Parameters. The following method will not be compiled. This is because, we are making the parameter restOfTheNumbers optional, but it appears before the required parameters "SN".

```

public static void ADDNumbers(int FN, int[] restOfTheNumbers = null, int SN)
{
    int result = FN + SN;
    // loop thru restOfTheNumbers only if it is not null otherwise
    // you will get a null reference exception
    if (restOfTheNumbers != null)
    {
        foreach (int i in restOfTheNumbers)
        {
            result += i;
        }
    }
    Console.WriteLine("Total = " + result.ToString());
}

```

Example to Understand Method Parameters Optional by Specifying Parameter Defaults:

In the below example, we are making Method Parameters Optional by Specifying Parameter Default value while declaring the parameter. As you can see, the ADDNumbers method takes three parameters. The first two parameters are mandatory and the last parameter is optional as we are specifying a default value for the third parameter.

```

using System;
namespace OptionalParameter
{
    class Program
    {

```

```

static void Main(string[] args)
{
    //Adding two Integer Numbers
    ADDNumbers(10, 20);
    //Adding Five Integer Numbers
    ADDNumbers(10, 20, new int[] { 30, 40, 50 });
    Console.ReadLine();
}
public static void ADDNumbers(int FN, int SN, int[] restOfTheNumbers = null)
{
    int result = FN + SN;
    //Loop through the restOfTheNumbers only if it is not null
    //else we will get runtime error
    if (restOfTheNumbers != null)
    {
        foreach (int i in restOfTheNumbers)
        {
            result += i;
        }
    }

    Console.WriteLine("Total = " + result.ToString());
}
}

```

Named Parameters in C#:

Named Parameters in C# allow developers to pass a method arguments with parameter names. For a better understanding, please have a look at the below code. In the following method, the parameters “b” & “c” are optional.

```

public static void Test(int a, int b = 10, int c = 20)
{
    Console.WriteLine("a = " + a);
    Console.WriteLine("b = " + b);
    Console.WriteLine("c = " + c);
}

```

When we invoke the above method as shown below 1 is passed as the argument for parameter a and 2 is passed as the argument for parameter b by default.

Test(1, 2);

My intention is to pass 2 as the argument for parameter c. To achieve this we can make use of named parameters, as shown below. Notice that, I have specified the name of the parameter for which value 2 is being passed.

Test(1, c: 2);

Example to Understand Named Parameters in C#:

The complete example is given below. In the below example, we are using named parameter while passing argument for the Method. With Named Parameter, the Order of passing the value is not important.

```

using System;
namespace NamedParameter
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Named Parameter while calling Method
            Test(1, 2); //a = 1 and b = 2 and c = 20 by default value
            Test(1, c: 2); //a = 1 and b = 10 by default and c = 2
            //Order is not Important with Named Parameter
            Test(b:1, c: 2, a:10);
            Console.ReadLine();
        }

        public static void Test(int a, int b = 10, int c = 20)
        {
            Console.WriteLine($"a = {a}, b = {b}, c= {c}");
        }
    }
}

```

Output:

```

a = 1, b = 2, c= 20
a = 1, b = 10, c= 2
a = 10, b = 1, c= 2

```

How to make Parameter Optional Using OptionalAttribute in C#:

Let us understand how to make method parameters optional by using the **OptionalAttribute** in C# that is present in **System.Runtime.InteropServices** namespace. Please have a look at the following function. Here, we decorate the third parameter with the Optional attribute which makes this parameter optional.

```

public static void ADDNumbers(int FN, int SN, [Optional] int[] restOfTheNumbers)
{
    int result = FN + SN;
    // loop thru restOfTheNumbers only if it is not null otherwise
    // you will get a null reference exception
    if (restOfTheNumbers != null)
    {
        foreach (int i in restOfTheNumbers)
        {
            result += i;
        }
    }
    Console.WriteLine("Total = " + result.ToString());
}

```

Here we are making the restOfTheNumbers optional by using the [Optional] attribute. Now, if we want to add just 2 numbers, then we can call the ADDNumbers method as follows.

ADDNumbers(10, 20);

On the other hand, if you want to add 3 or more numbers, then you can invoke the method ADDNumbers() as

follows.

```
ADDNumbers(10, 20, new int[] { 30, 40 });
```

The complete example is given below.

```
using System;
using System.Runtime.InteropServices;
namespace OptionalParameter
{
    class Program
    {
        static void Main(string[] args)
        {
            ADDNumbers(10, 20);
            ADDNumbers(10, 20, new int[] { 30, 40, 50 });

            Console.ReadLine();
        }
        public static void ADDNumbers(int FN, int SN, [Optional] int[] restOfTheNurr
        {
            int result = FN + SN;
            // loop thru restOfTheNumbers only if it is not null otherwise
            // you will get a null reference exception
            if (restOfTheNumbers != null)
            {
                foreach (int i in restOfTheNumbers)
                {
                    result += i;
                }
            }
            Console.WriteLine("Total = " + result.ToString());
        }
    }
}
```

Output:

```
Total = 30
Total = 150
```

In the next article, I am going to discuss [How to Create and use Indexers in C#](#) with Examples. Here, in this article, I try to explain **How to Make Optional Parameters in C#** with examples. And I hope you enjoy this How to Make Method Parameters Optional in C# with Examples article.